



Production Information Systems

Topic : Object-oriented analysis and design

Chapter 5 : Unified Modeling Language

ISE Department
Year 4/ Level 7

Instructor : Dr. Jomana Bashatah



Outline

1. Introduction
2. Object-Oriented Design (OOD) concept
3. UML design formalisms
4. Architecture Design using UML
A Use Case Diagram
5. Sequence Diagram
6. Activity Diagram
7. State Chart Diagrams
8. Class Diagram

1.Introduction

- The previous chapter emphasized a particular process for the conceptual-level design of an information system. This process includes an activity design component (IDEF0 or DFD)
- However, since the early 1990s, new approaches have emerged and have achieved some level of success due in part to the introduction of new modeling frameworks and new enhancements that are not part of the traditional framework.
- As example, we cite the Unified Modeling Language (*Jacobson et al., 1999; Roques, 2001; Rumbaugh et al., 1999*).

1.Introduction

- One of the most significant of the newer approaches owes its genesis to developments in software engineering, namely ***object-oriented (OO)*** software development.
- The object-oriented approach stresses the encapsulation of data and procedures within segments of software code called objects.
- Instead of separating programming code from data files used by the code, the two are bound together in a module, **the object**. Objects are instantiations of classes, which are generic templates of the objects. The object has the characteristics of the class from which it is created. For example, many of the conceptual underpinnings of Microsoft Access and its supporting programming language, Visual Basic for Applications (VBA), are based on classes and objects.

1.Introduction

- A specific set of object-oriented design tools that evolved from the trend in object thinking is known as ***Unified Modeling Language (UML)***.
- UML, like the IDEF framework, is a set of tools developed to assist analysts in uncovering the important features of a design project, finally arriving at a set of models that will be used to design, document, and implement the project, whether it is an information system or other software development project.
- In database design projects, UML is often thought of as a tool for object-oriented databases. As we shall see, the modeling components of UML have special features that complement the object-oriented paradigm. However, even if the target database management system (DBMS) is a relational database, the UML design tools can be used.

2.Object-Oriented design concept

To introduce UML modeling formalisms, some background in object-oriented terminology is required. In traditional programming, code is written as functions that operate on input data and (usually) provide some output to a program. Functions are called from a main program in some sequence to accomplish the work required by the program.

Among the functions required may be one to open a database or perform a structured query language (SQL) on the contents of a database.

The overall flow of control lies in the main program, which calls the required functions sequentially. This form of program design has traditionally been called “structured programming.”

2.Object-Oriented design concept

- **object-oriented design** helps create autonomous structures (the objects) that represent real-world entities and have the ability to interact with other objects.
- The motivation for OOD methods in the software development community is economic.
- If software can be designed as objects, the objects can be used and reused to design systems.
- The programmer no longer has to deal with writing detailed code. Rather, the system is designed by making appropriate connections between objects that must communicate with each other.
- This reduces development time and enhances system reliability.

2.Object-Oriented design concept

There are a number of concepts in object-oriented design that, when taken together, define whether or not an application is truly object-oriented.

Here we briefly review some of these concepts:

- The concept of class is an important feature of object-oriented design: A ***class*** is a collection of objects with shared structure and behavior.
- In OOD, ***the object*** is a specific individual member of a class. ***Class*** is the generic description of the collection of objects.

So, for example, class would be the abstract concept of a tree, whereas a specific tree in your backyard, a member of that class, is an object.

2.Object-Oriented design concept

The object is an instance of the class and has the attributes of the class of which it is an instance.

One of the attributes is its object ID (OID), which is a unique identification of the object. No two objects of the class can have the same OID. The OID has something in common with the concept of a social security number for the class of humans called U.S. residents.

The concept of ***inheritance*** has a special meaning in OOD. Inheritance is used to describe a class and its subclasses.

For example: the superclass SENSOR has two subclasses, BINARY and ANALOG. BINARY and ANALOG are subclasses of SENSOR, and they inherit some methods and attributes from the superclass. However, there may be some methods and attributes of the superclass that are not inherited by the subclass.

2.Object-Oriented design concept

An object, on the other hand, is a realization of **a class**. An object of a class always has all the attributes and methods of the class.

For example: a cat is a subclass of another abstraction, animals (which is itself a class). However, the cat in your house, Felix, is a unique individual and a real thing. Felix is an object.

An **attribute** is a characteristic of an object and class.

Example:

The cat class has an **attribute** color. **The object** in your house, Felix the cat, has an attribute color with a **value** black. **Classes** have **attributes**, but no **values**. **Objects** of a **class** have the **attributes** of the class and **values** for those attributes.

2.Object-Oriented design concept

Objects operate under their own thread of control. An object evolves through a series of ***states***, which describe the dynamic behavior of the object.

The states are a representation of the various phases the object passes through during its operating cycle.

Consider, for example, Felix the cat is usually in the sleeping state, but when awake he could be in the eating state, moving state, and so on. These analogies are quite general and are a part of the description of an object's behavior.

2.Object-Oriented design concept

An object has a set of software functions that it can execute. The functions are often associated with a state of the object, but this is not necessary.

The important thing is that an object can use these functions whenever it is appropriate. In the OO world, these functions are called ***methods***.

For example: temperature sensor has a method called `get_value( )`. In this method, it simply assigns the current value of the temperature reading to a variable. Perhaps this is done periodically — for example, every second. We say that the sensor ***invokes*** the method every second. An object's methods may be invoked by the object itself.

Felix the cat has methods that may be self-invoked. For example, the meow method.

2.Object-Oriented design concept

- Object-oriented programming was a motivating force for the development of design architecture tools for OO design projects.
- This design architecture is known as Unified Modeling Language.
- Though based on OO concepts, UML is finding use in software and database development even when it is not based on OO technologies, like C++, Java, and object-oriented databases.
- Therefore, we present UML as an emerging alternative to traditional conceptual modeling techniques and illustrate its application for object-oriented design as well as more traditional information system design projects.

3.UML design formalisms

UML supports nine application specification design models in the form of UML diagrams. They can be classified as either structural diagrams or behavioral diagrams.

The ***structural diagrams*** define the static relationships among components of the architecture.

The ***behavioral diagrams*** define the dynamics of components of the design.

- The categorization of diagrams is as follows:
 - ✓ ***Structural:*** **Class diagrams, object diagrams, component diagrams and deployment diagrams**
 - ✓ ***Behavioral:*** **Use case diagrams, sequence diagrams, collaboration diagrams, state chart diagrams and activity diagrams**

3.UML design formalisms

Some of these diagrams are more closely associated with software development than information system design. We shall focus on the most relevant design diagrams. However, a brief definition of each diagram type is as follows :

- 1. *Class diagram*:** A graphic representation of a collection of model elements, such as classes, types, and relationships. It includes a conceptual model of database design elements.
- 2. *Object diagram*:** Shows objects and their relationships at a point in time. It is a special case of a class diagram showing instances of classes and their relationships.

3.UML design formalisms

3. *Component diagram:* A component is a physical piece of implementation of a system, such as a module of software code. Components can be connected to other components through interfaces. A component diagram is a diagram that shows the organization and dependencies among component types.

4. *Deployment diagram:* When components are used in a system, they are deployed as component instances. A deployment diagram shows the component instances and the objects that are associated with them. Therefore, a deployment diagram shows instances of a component, while a component diagram shows the definition of component types.

3.UML design formalisms

5. *Use case diagram:* Use case is a function that the system can perform when it interacts with outside actors. The concept of an “actor” is an abstraction of a person or thing outside the system that interacts directly with the system — for example, a database user. The use case diagram shows the functional interaction between actors and the system when performing a user function.

6. *Sequence diagram:* A sequence diagram shows the interaction of the actor with the objects in the system and the timing of the interactions. It gives a dynamic view by time-sequencing the flow of messages among objects.

3.UML design formalisms

7. *Collaboration diagram:* A collaboration is an arrangement of objects and links that interact to implement a behavior, such as a use case. The collaboration diagram illustrates that interaction.

8. *State chart diagram:* A state machine is a sequence of states and transitions between states that an object goes through in response to events in its life cycle. A state chart diagram shows the state machine, including nested component states, in a simple diagrammatic form. State chart diagrams depict the dynamics of objects.

9. *Activity diagram:* An activity is an execution of a process, which could be within the system or a real-world function that interacts with the system. An activity diagram shows a graph that models the activity. Activity diagrams are very useful in modeling business operations and workflows.

4. Architecture design using UML

A use Case Diagram

A ***use case diagram*** is a description of how users interact with the system.

To develop a use case diagram, the analyst must identify three entities:

- The system boundary and interfaces
- The actors who use the system
- The use cases (i.e., the functions that the actor calls upon the system to perform)

4. Architecture design using UML

A use Case Diagram

The figure below shows a use case diagram for a hypothetical online theater ticket sales system.

The diagram shows the three components. The use case is depicted as an oval with the use case name within the oval, in this example “make sale.” The system is shown as a box that makes explicit the boundaries between the use case and its environment.

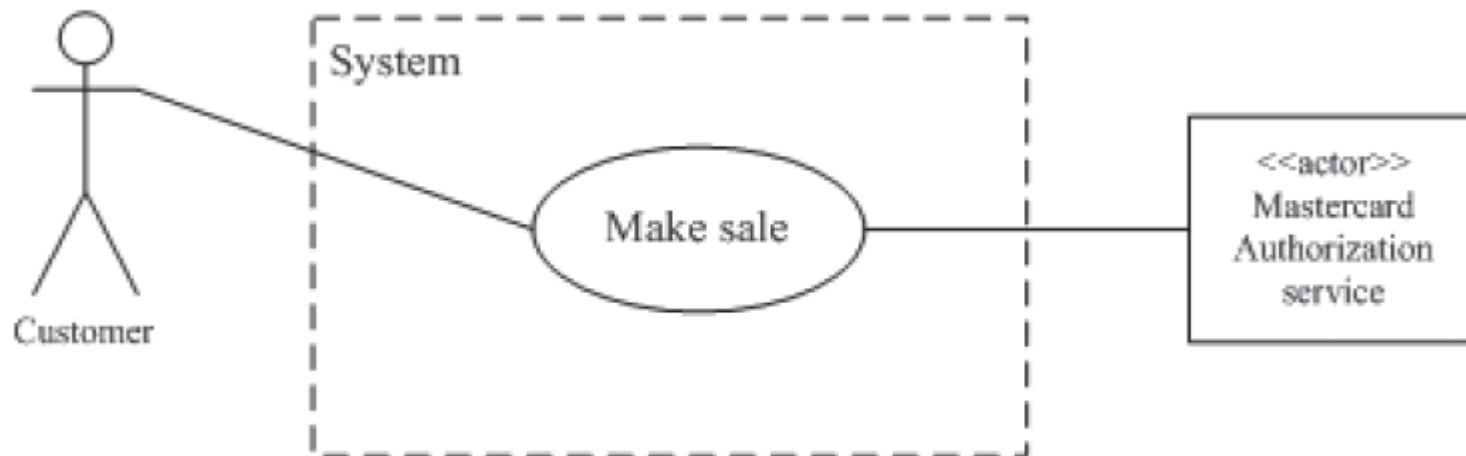


Figure 9.3 Make sale use case diagram.

4. Architecture design using UML

A use Case Diagram

The ***system*** is that which is being designed. The analysts use UML models to document their design. For the purposes of this course, the system is usually an information system. An information system can have many components, of which the database is perhaps the most important.

But, in addition, it may have sensors and data acquisition devices, such as bar code scanners. It may also have intelligent agents that serve as intermediaries between the user and the database.

4. Architecture design using UML

A use Case Diagram

An **agent** is a software component that is situated in some environment, can perceive its environment, and is capable of autonomous action to meet the design objectives of the application.

If you have called a travel company to obtain a train reservation, you probably talked with the reservation agent which can be an intelligent software agent (Julie) with sophisticated voice recognition capabilities that allow it to hold a conversation with a customer. For the company, Julie is part of the software system that is the focus of the design and would be included within the system box on a use case diagram.

4. Architecture design using UML

A use Case Diagram

Example :

In the case of online theater ticket sales, we assume the existence of a Web server with forms and software to serve as an interface. We assume VBScript code that accesses the database and communicates with the actors. We will refer to this interface as “Ticket Agent.”

Ticket Agent is part of the software system for theater ticket sales and would be included within the system box on a use case diagram.

4. Architecture design using UML

A use Case Diagram

- Finally, there are the actors. The **actors** are external to the system but interact with it.
- A common way of showing an actor when it represents a person type is to use a stick figure, with the role of the actor shown below the stick figure.
- This is illustrated as the “customer” in the Figure before.
- An actor should be described in terms of the “role” it plays.
- The actor may represent many different groups of people, but it is the logical role it plays when interfacing with the system that is important.

4. Architecture design using UML

A use Case Diagram

Example :

suppose that Julie were replaced with a real ticket agent. The customer would telephone the real ticket agent and ask for a reservation.

The ticket agent would then use the computer keyboard to interface with the system in order to key in the reservation for the customer.

The customer would not interact with the system.

For this case, the actor would be the ticket agent, not the customer.

So the use case diagram considers the actors that are *directly interfacing* with the system.

4. Architecture design using UML

A use Case Diagram

Example :

Another way to represent an actor, particularly if it is not a person, is by using a box with the word <<actor>> in guillemets.

In the case of online theater ticket sales, a credit card is usually used to buy the ticket.

One of the actors that interact with the system is the authorization service of the credit card company.

The credit card authorization service is another automated system. However, to the theater ticket sales system, it is external and, therefore, an actor.

4. Architecture design using UML

A use Case Diagram

- The use case diagram tells a story. One of the uses of the system is to make a sale.
- There are two actors (external entities) involved. One is the customer and the other is the credit authorization entity. When documenting a use case, it is important to write an explanation of the use case that will help convey the message of the diagram. This document can be in the form of a **text file** that describes the activities that take place in executing the use case. These descriptions are called **scenarios**.
- A use case diagram can contain several use cases, depending on the various functions to be supported by the system.

4. Architecture design using UML

A use Case Diagram

Another use case has been added in this Figure. This is the case in which customers go online and query the system for the availability of seats for a performance. This is often done without following through to purchase a seat. For this use case, the customer is the only actor involved.

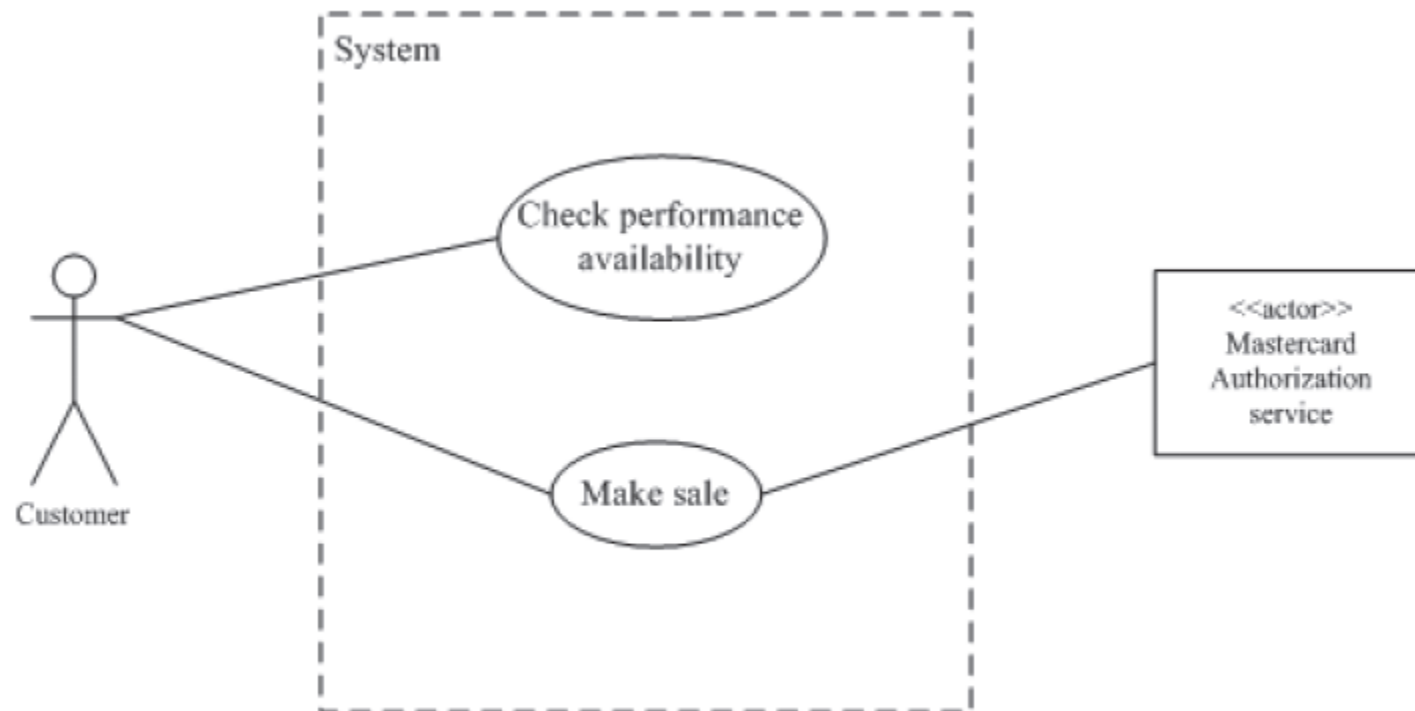


Figure 9.4 Theatre ticket use case diagram.

A use Case Diagram

Case study

The scenario is to “Confirm Validity of Shipment”. When a delivery truck arrives at the receiving dock, the receiving clerk is responsible for clearing the material for unloading. The clerk first compares the paperwork that comes with the shipment with the enterprise’s purchase order that is referenced on the bill of lading. Therefore, two required information inputs to this function are the bill of lading and the corresponding purchase order. If the material on the shipment bill of lading corresponds to the purchase order, the shipment is accepted. If no such purchase order exists, the shipment is refused. This procedure is defined as the “material receiving procedure.” The steps of this procedure, as defined by the management of the enterprise, control the manner in which the function is performed.

A use Case Diagram

Case study

Two actors interface with the system (database) when using the purchase order in various use cases:

The purchasing agent **creates** the purchase order. This includes all of the header information in the Purchase Order table and the detail information in the PO_Detail table.

After being created, the purchase order **is printed** and sent to the vendor.

No further use is made of the purchase order information until material is delivered. At that time, the receiving clerk compares the shipment information to the purchase order (**Confirm** Validity of Shipment) and prepares a receiving report or otherwise informs the purchasing agent of the delivery against the purchase order (**Notify** Purchasing Agent).

The purchasing agent will then change the purchase order balances by subtracting the amount received by line item from the current balance (**Update** the PO balances).

Finally, when all line items have been delivered, the purchasing agent will **close** the purchase order.

